

实 验 报 告

一、实验基本信息

实验名称: pix2pix 实验

姓名: 赵宸

学号: 2302020530

班级: 化学 235

实验日期: 2025. 7. 27

实验平台: Conda/Pycharm/终端

二、实验目的

理解图像到图像翻译的基本原理

掌握 Pix2Pix 生成对抗网络的结构

能够使用开源实现进行训练与图像翻译测试

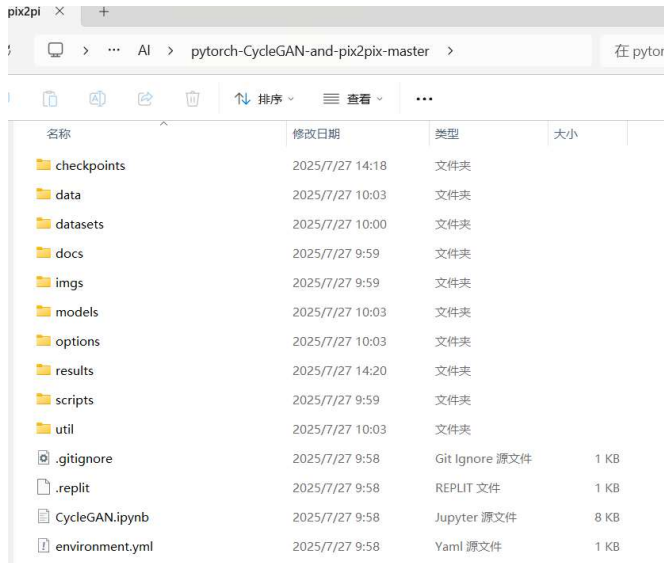
熟悉使用小型数据集在本地环境训练图像生成模型

三、实验环境配置

由于之前已经配置好了 Spider 环境包含 torchvision, torch 等, 所以这里没有新建新的虚拟环境, 直接在原环境中进行了实验, 并安装了 visdom, wandb 等其他依赖的第三方库

四、实验步骤与过程记录

先下载本地模型与数据集，并放到目录中



然后进入到 conda prompt 中进行代码运行，进入到 Spider 虚拟环境中，然后 cd 到当前的目录下，执行代码 `python train.py ./datasets/facades -name facades_pix2pix -model pix2pix -direction BtoA -n_epochs 20 -n_epochs_decay 0 -batch_size 1 -gpu_ids -1 display_id -1 -save_epoch_freq 1`，开始训练模型

```
conda prompt
(pytorch) C:\Users\user>cd pytorch-CycleGAN-and-pix2pix-master
(pytorch) C:\Users\user\pytorch-CycleGAN-and-pix2pix-master>python train.py ./datasets/facades -name facades_pix2pix -model pix2pix -direction BtoA -n_epochs 20 -n_epochs_decay 0 -batch_size 1 -gpu_ids -1 display_id -1 -save_epoch_freq 1
(epoch: 17, iters: 200, time: 1.710, data: 0.001) G_GAN: 1.821 G_L1: 31.038 D_real: 0.019 D_fake: 0.983
(epoch: 17, iters: 300, time: 1.709, data: 0.002) G_GAN: 3.235 G_L1: 31.764 D_real: 0.060 D_fake: 0.156
(epoch: 17, iters: 400, time: 2.005, data: 0.001) G_GAN: 1.558 G_L1: 29.070 D_real: 0.586 D_fake: 0.399
saving the model at the end of epoch 17, iters 6800
End of epoch 17 / 20 Time Taken: 717 sec
learning rate 0.0002000 -> 0.0002000
(epoch: 18, iters: 100, time: 1.733, data: 21.217) G_GAN: 1.277 G_L1: 27.480 D_real: 0.654 D_fake: 0.231
(epoch: 18, iters: 200, time: 1.724, data: 0.001) G_GAN: 1.198 G_L1: 26.171 D_real: 1.007 D_fake: 0.260
(epoch: 18, iters: 300, time: 1.706, data: 0.001) G_GAN: 2.068 G_L1: 36.763 D_real: 0.009 D_fake: 0.193
(epoch: 18, iters: 400, time: 1.986, data: 0.001) G_GAN: 2.199 G_L1: 38.206 D_real: 0.007 D_fake: 0.381
saving the model at the end of epoch 18, iters 7200
End of epoch 18 / 20 Time Taken: 716 sec
learning rate 0.0002000 -> 0.0002000
(epoch: 19, iters: 100, time: 1.665, data: 20.358) G_GAN: 3.449 G_L1: 28.609 D_real: 0.149 D_fake: 0.148
(epoch: 19, iters: 200, time: 1.613, data: 0.002) G_GAN: 1.919 G_L1: 31.672 D_real: 0.373 D_fake: 0.365
(epoch: 19, iters: 300, time: 1.732, data: 0.001) G_GAN: 3.379 G_L1: 44.059 D_real: 0.005 D_fake: 0.079
(epoch: 19, iters: 400, time: 2.009, data: 0.001) G_GAN: 2.159 G_L1: 41.034 D_real: 0.011 D_fake: 0.525
saving the model at the end of epoch 19, iters 7600
End of epoch 19 / 20 Time Taken: 707 sec
learning rate 0.0002000 -> 0.0002000
(epoch: 20, iters: 100, time: 1.599, data: 36.738) G_GAN: 2.250 G_L1: 30.053 D_real: 3.604 D_fake: 0.150
(epoch: 20, iters: 200, time: 1.781, data: 0.001) G_GAN: 1.273 G_L1: 25.001 D_real: 0.945 D_fake: 0.395
(epoch: 20, iters: 300, time: 1.815, data: 0.001) G_GAN: 2.166 G_L1: 40.567 D_real: 0.057 D_fake: 0.150
(epoch: 20, iters: 400, time: 2.115, data: 0.002) G_GAN: 2.042 G_L1: 40.775 D_real: 0.046 D_fake: 0.172
saving the model at the end of epoch 20, iters 8000
End of epoch 20 / 20 Time Taken: 717 sec
```

在长达三个小时的训练之后，模型训练完毕，可以在后续中使用

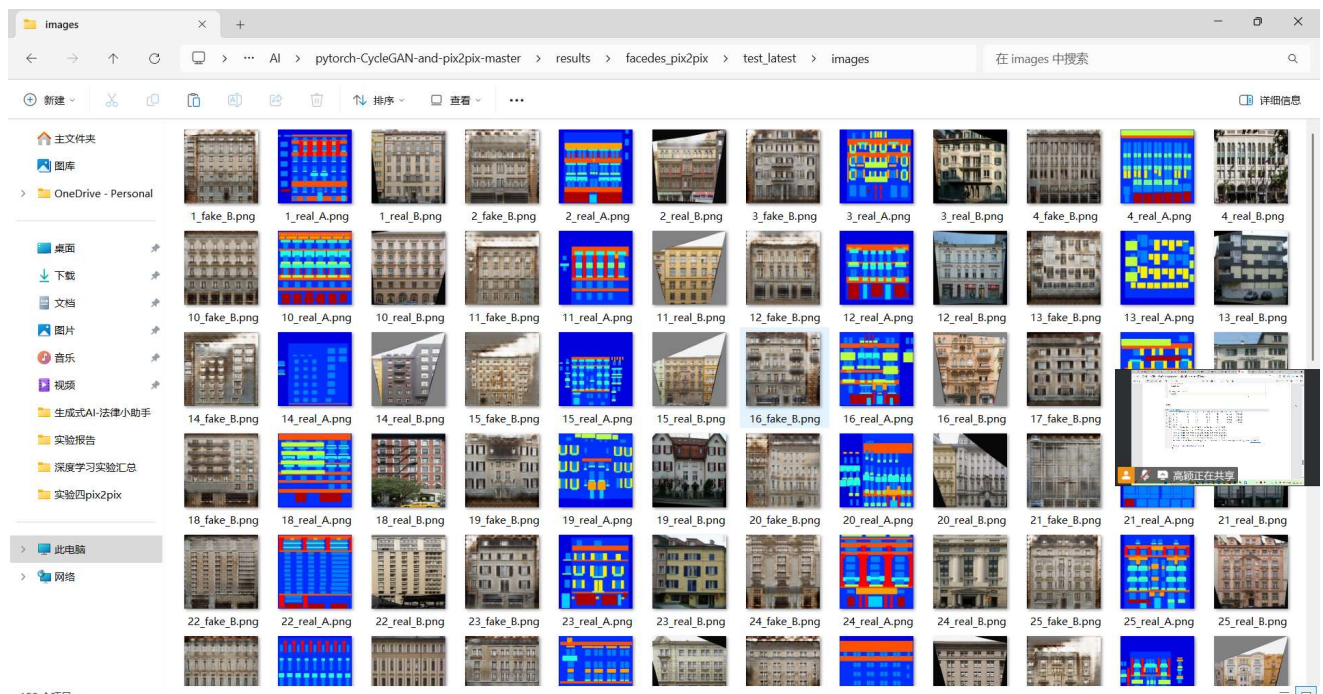
然后就可以进行模型测试了，运行 `python test.py -`

`dataroot ./datasets/facades -name facades_pix2pix -model pix2pix -direction BtoA -gpu_ids -1`

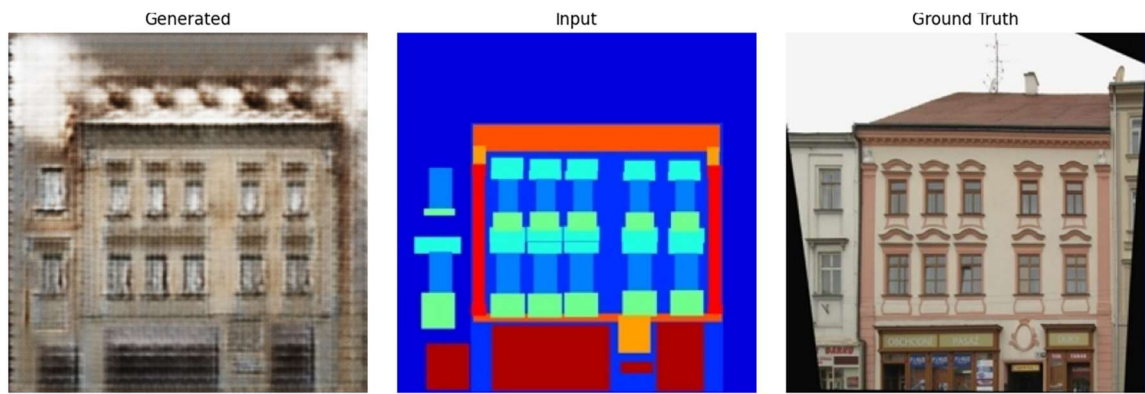
```
Anaconda Prompt

output_nc: 3
phase: test
preprocess: resize_and_crop
results_dir: ./results/
serial_batches: False
suffix:
use_wandb: False
verbose: False
wandb_project_name: CycleGAN-and-pix2pix
----- End -----
dataset [AlignedDataset] was created
initialize network with normal
model [Pix2PixModel] was created
loading the model from ./checkpoints/facedes_pix2pix/latest_net_G.pth
----- Networks initialized -----
[Network G] Total number of parameters : 54.414 M
-----
creating web directory ./results/facedes_pix2pix/test_latest
processing (0000)-th image... ['./datasets/facades\\test\\1.jpg']
processing (0005)-th image... ['./datasets/facades\\test\\103.jpg']
processing (0010)-th image... ['./datasets/facades\\test\\12.jpg']
processing (0015)-th image... ['./datasets/facades\\test\\17.jpg']
processing (0020)-th image... ['./datasets/facades\\test\\21.jpg']
processing (0025)-th image... ['./datasets/facades\\test\\26.jpg']
processing (0030)-th image... ['./datasets/facades\\test\\30.jpg']
processing (0035)-th image... ['./datasets/facades\\test\\35.jpg']
processing (0040)-th image... ['./datasets/facades\\test\\4.jpg']
processing (0045)-th image... ['./datasets/facades\\test\\44.jpg']
```

测试完成，结果被成功保存在了 results 目录当中



然后对结果进行可视化分析，分析生成对抗网络的效果如何



生成结果如图所示，源代码粘贴在下面

```
import matplotlib.pyplot as plt
import os
from PIL import Image

result_dir='D:/AI/pytorch-CycleGAN-and-pix2pix-
master/results/facedes_pix2pix/test_latest/images'
imgs=sorted(os.listdir(result_dir))
n=1
start_idx=n*3
if start_idx+2<len(imgs):
    fig,axs=plt.subplots(1,3,figsize=(12,4))

    axs[0].imshow(Image.open(os.path.join(result_dir,imgs[start_idx])))
    axs[0].set_title('Generated')

    axs[1].imshow(Image.open(os.path.join(result_dir,imgs[start_idx+1])))
    axs[1].set_title('Input')

    axs[2].imshow(Image.open(os.path.join(result_dir,imgs[start_idx+2])))
    axs[2].set_title('Ground Truth')

    for ax in axs:
        ax.axis('off')
```



```
plt.tight_layout()

plt.show()

else:

    print('你选择的编号超出了图片数量范围')
```

然后改变了数据流动的方向，从 AtoB，进行了重新的训练

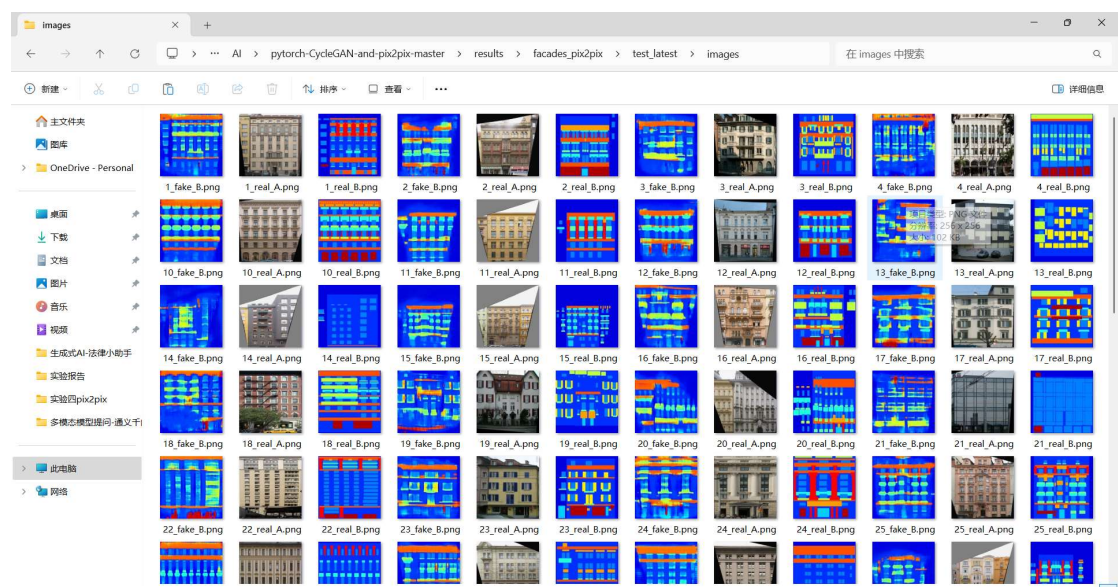
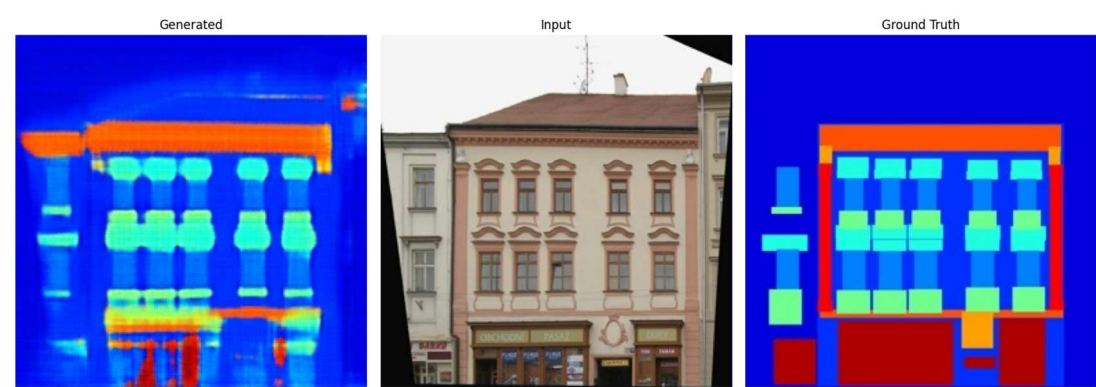
更改了 direction 的参数，进行了新一轮的训练，保存了模型与权重

```
Anaconda Prompt
(epoch: 15, iters: 300, time: 1.709, data: 0.002) G_GAN: 2.645 G_L1: 19.602 D_real: 0.040 D_fake: 0.308
(epoch: 15, iters: 400, time: 2.087, data: 0.001) G_GAN: 2.493 G_L1: 23.465 D_real: 1.213 D_fake: 0.007
saving the model at the end of epoch 15, iters 6000
End of epoch 15 / 20      Time Taken: 728 sec
learning rate 0.0002000 -> 0.0002000
(epoch: 16, iters: 100, time: 1.814, data: 32.329) G_GAN: 5.130 G_L1: 22.131 D_real: 0.070 D_fake: 0.803
(epoch: 16, iters: 200, time: 1.769, data: 0.001) G_GAN: 3.230 G_L1: 19.461 D_real: 0.100 D_fake: 0.124
(epoch: 16, iters: 300, time: 1.657, data: 0.001) G_GAN: 3.341 G_L1: 33.035 D_real: 0.004 D_fake: 0.057
(epoch: 16, iters: 400, time: 2.092, data: 0.002) G_GAN: 3.801 G_L1: 38.214 D_real: 0.002 D_fake: 0.043
saving the model at the end of epoch 16, iters 6400
End of epoch 16 / 20      Time Taken: 738 sec
learning rate 0.0002000 -> 0.0002000
(epoch: 17, iters: 100, time: 1.651, data: 30.851) G_GAN: 3.572 G_L1: 37.441 D_real: 0.962 D_fake: 0.006
(epoch: 17, iters: 200, time: 1.816, data: 0.001) G_GAN: 3.395 G_L1: 31.156 D_real: 0.001 D_fake: 0.458
(epoch: 17, iters: 300, time: 1.789, data: 0.001) G_GAN: 4.619 G_L1: 28.865 D_real: 0.013 D_fake: 0.013
(epoch: 17, iters: 400, time: 1.971, data: 0.001) G_GAN: 5.341 G_L1: 21.203 D_real: 0.003 D_fake: 0.471
saving the model at the end of epoch 17, iters 6800
End of epoch 17 / 20      Time Taken: 762 sec
learning rate 0.0002000 -> 0.0002000
(epoch: 18, iters: 100, time: 1.883, data: 34.570) G_GAN: 4.734 G_L1: 19.849 D_real: 0.035 D_fake: 0.535
(epoch: 18, iters: 200, time: 1.689, data: 0.001) G_GAN: 6.336 G_L1: 35.470 D_real: 0.003 D_fake: 0.002
(epoch: 18, iters: 300, time: 1.886, data: 0.000) G_GAN: 3.369 G_L1: 47.518 D_real: 0.000 D_fake: 0.225
(epoch: 18, iters: 400, time: 1.999, data: 0.001) G_GAN: 3.369 G_L1: 31.797 D_real: 0.014 D_fake: 0.291
saving the model at the end of epoch 18, iters 7200
End of epoch 18 / 20      Time Taken: 741 sec
learning rate 0.0002000 -> 0.0002000
(epoch: 19, iters: 100, time: 1.821, data: 34.691) G_GAN: 4.081 G_L1: 30.974 D_real: 0.001 D_fake: 0.026
(epoch: 19, iters: 200, time: 1.612, data: 0.003) G_GAN: 3.808 G_L1: 42.645 D_real: 0.000 D_fake: 0.191
(epoch: 19, iters: 300, time: 1.730, data: 0.001) G_GAN: 3.088 G_L1: 30.691 D_real: 2.739 D_fake: 0.003
(epoch: 19, iters: 400, time: 2.181, data: 0.002) G_GAN: 2.109 G_L1: 11.784 D_real: 0.526 D_fake: 0.068
saving the model at the end of epoch 19, iters 7600
End of epoch 19 / 20      Time Taken: 755 sec
learning rate 0.0002000 -> 0.0000000
(epoch: 20, iters: 100, time: 1.748, data: 40.568) G_GAN: 2.201 G_L1: 35.306 D_real: 0.044 D_fake: 0.179
(epoch: 20, iters: 200, time: 1.703, data: 0.001) G_GAN: 0.980 G_L1: 18.572 D_real: 0.004 D_fake: 0.618
(epoch: 20, iters: 300, time: 1.407, data: 0.002) G_GAN: 3.641 G_L1: 34.313 D_real: 0.004 D_fake: 0.045
(epoch: 20, iters: 400, time: 2.129, data: 0.001) G_GAN: 4.765 G_L1: 42.980 D_real: 0.002 D_fake: 0.016
saving the model at the end of epoch 20, iters 8000
End of epoch 20 / 20      Time Taken: 720 sec
```

然后运行 test.py，同样更改了一下 direction 的参数

```
serial_batches: False
suffix:
use_wandb: False
verbose: False
wandb_project_name: CycleGAN-and-pix2pix
----- End -----
dataset [AlignedDataset] was created
initialize network with normal
model [Pix2PixModel] was created
loading the model from ./checkpoints/facades_pix2pix/latest_net_G.pth
----- Networks initialized -----
[Network G] Total number of parameters : 54.414 M
-----
creating web directory ./results/facades_pix2pix/test_latest
processing (0000)-th image... ['./datasets/facades/test/1.jpg']
processing (0005)-th image... ['./datasets/facades/test/103.jpg']
processing (0010)-th image... ['./datasets/facades/test/12.jpg']
processing (0015)-th image... ['./datasets/facades/test/17.jpg']
processing (0020)-th image... ['./datasets/facades/test/21.jpg']
processing (0025)-th image... ['./datasets/facades/test/26.jpg']
processing (0030)-th image... ['./datasets/facades/test/30.jpg']
processing (0035)-th image... ['./datasets/facades/test/35.jpg']
processing (0040)-th image... ['./datasets/facades/test/4.jpg']
processing (0045)-th image... ['./datasets/facades/test/44.jpg']
```

结果成功保存了，并对结果进行可视化



五、实验结果与分析

实验分别进行了从 A 到 B 的生成和从 B 到 A 的生成，在我看来，AtoB 的效果更好，建筑的一些基本结构都能比较不错的生成出来，而且颜色也基本还原，看起来从建筑结构到抽象图的效果还是很不错的，相比较于 BtoA，虽然整体结构也有，但是细节还是不够清晰，有一点空间扭曲的感觉，说明从抽象图到具体建筑的能力还是没有那么强的。

总结下来，AtoB 的生成效果更接近 Ground Truth，在这个任务当中，结构与语义的还原度较高，而 BtoA 的重建难度更大一些，生成效果比较模糊，结构不太清晰，还原度低。这反映出生成对抗网络在从具体到抽象的转换中更容易学习到规律，反之，从抽象到具体则更需要更强的模块重建能力，后面可以对 BtoA 进行优化，比如添加超分辨率模块，增加训练次数等来提高效果

六、总结

掌握了生成对抗网络的基本原理，并完成了 pix2pix 的生成对抗网络实验，巩固了命令行的各种操作与代码运行，并试验了不同的 direction 对生成对抗网络的结果的影响，并通过可视化结果，直观的比较生成效果之间的差异，顺利完成了实验，并完成了一些拓展任务，自行排除了一些程序无法运行的地方